FPT UNIVERSITY

FSB Institute of Management and Technology

(Vien Quan tri & Cong nghe FSB, Truong Dai hoc FPT)

COURSE

DDM501

AI in Production: From Models to Systems

Duration: 4 weeks · Weight: 40%

FINAL PROJECT REPORT

Sentiment Analysis Service

End-to-End ML System: From Problem Definition to Production Deployment

Topic: NLP – Topic 8 – Sentiment Analysis Service

Dataset: SemEval-2014 · tweet_eval_irony_v1 · multilingual_sentiment_v1

ML Model: XLM-RoBERTa + PEFT/LoRA (Sentiment & Sarcasm Adapters)

TEAM MEMBERS

#	Full Name	Student ID	Responsibility
1	Duong Hong Quan	MSA23236	Monitoring, DevOps / CI-CD
2	Pham Duc Long	MSA23233	Backend API, Experiment Tracking
3	Do Quoc Trung	MSA23231	ML Pipeline, Data Validation

Instructor: Huynh Cong Viet Ngu

Submission: April 2026

Academic Year: 2025 – 2026

Abstract

This report documents the complete lifecycle of the **Sentiment Analysis Service**, a production-grade ML system developed as the capstone project for the DDM501 course. The system performs real-time sentence-level sentiment classification (positive / negative / neutral) as well as Aspect-Based Sentiment Analysis (ABSA). The core model is `xlm-roberta-base`, fine-tuned via Parameter-Efficient Fine-Tuning (PEFT/LoRA) to produce separate lightweight adapters for Sentiment Analysis and Sarcasm Detection, enabling multilingual support across English and Vietnamese.

The service is exposed through a RESTful FastAPI application, containerised with Docker, orchestrated via Docker Compose, and equipped with a full observability stack (Prometheus + Grafana), experiment tracking (MLflow), automated data validation (DVC), a robust test suite, and a GitHub Actions CI/CD pipeline. A dedicated `/explain` endpoint delivers token-level SHAP attributions, fulfilling the Responsible AI explainability requirement.

Contents

1 Problem Definition & Requirements (10%) 4
- 1.1 Problem Statement & Business Context . . . 4
- 1.2 User Requirements & Use Cases . . . 4
- 1.3 Success Metrics . . . 5
- 1.4 Scope & Constraints . . . 5

2 System Design & Architecture (15%) 5
- 2.1 High-Level Architecture . . . 6
- 2.2 Component Design & Responsibilities . . . 6
- 2.3 Data Flow . . . 7
- 2.4 Technology Stack Justification . . . 8

3 Implementation (40%) 9
- 3.1 ML Pipeline (15%) . . . 9
  - 3.1.1 Datasets & Machine Learning Models . . . 9
  - 3.1.2 Reproducible DVC Pipeline . . . 10
  - 3.1.3 Experiment Tracking with MLflow . . . 11
  - 3.1.4 Model Evaluation & Fine-tuning Results . . . 12
- 3.2 Deployment (15%) . . . 13
  - 3.2.1 REST API Design . . . 13
  - 3.2.2 Docker Containerisation . . . 15
  - 3.2.3 Docker Compose Orchestration . . . 16
- 3.3 Monitoring (10%) . . . 16
  - 3.3.1 Prometheus Metrics . . . 16
  - 3.3.2 Grafana Dashboards . . . 17
  - 3.3.3 Alerting Rules . . . 17

4 Testing & CI/CD (15%) 17
- 4.1 Test Strategy . . . 17
  - 4.1.1 Integration Test Examples . . . 18
- 4.2 CI/CD Pipeline . . . 18

5 Responsible AI (10%) 19
- 5.1 Model Explainability with SHAP . . . 19
  - 5.1.1 Technical Implementation . . . 19
- 5.2 Fairness Analysis . . . 22
- 5.3 Data Privacy Considerations . . . 23
- 5.4 Ethical Implications . . . 23

6 Documentation (10%) **23**
6.1 Repository Structure 23
6.2 API Documentation 24
6.3 Setup & Deployment Guide 24
6.4 Code Quality 25

7 Conclusion **25**

A Full Dependency List **26**

B Individual Contribution **26**

1 Problem Definition & Requirements (10%)

1.1 Problem Statement & Business Context

Customer feedback represents one of the most valuable yet under-utilised assets of any business. Manual review of large volumes of reviews, social-media posts, and support tickets is both expensive and slow.

The **Sentiment Analysis Service** addresses this gap by providing an automated, REST-accessible system that:

a) classifies the *overall* sentiment of a text snippet (positive · negative · neutral);

b) extracts *aspect-level* sentiments (food, service, ambiance, price, location, general) for fine-grained business insight;

c) provides *SHAP-based token explanations* so that end-users can trust and audit model decisions.

The primary dataset used for offline evaluation is the **SemEval-2014 Task 4 Restaurant Reviews** corpus, which provides sentence-level and aspect-level sentiment annotations.

1.2 User Requirements & Use Cases

Table 1: Functional Requirements

ID	Priority	Requirement
FR-01	Must Have	Single-text sentiment prediction via `POST /predict` endpoint.
FR-02	Must Have	Aspect-Based Sentiment Analysis (ABSA) returned alongside overall prediction.
FR-03	Must Have	Batch processing of CSV files via `POST /batch_predict`.
FR-04	Must Have	Model explainability (SHAP values) via `POST /explain` endpoint.
FR-05	Must Have	Health-check endpoint for load-balancer liveness probes.
FR-06	Should Have	Language guard: reject non-English inputs with HTTP 400.
FR-07	Should Have	Prometheus metrics endpoint `GET /metrics` for observability.
FR-08	Could Have	Angular-based frontend chatbot interface.

Table 2: Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	P95 end-to-end latency < 200 ms for single-text prediction.
NFR-02	Reliability	Service uptime $\geq 99.9\%$.
NFR-03	Scalability	Stateless API; horizontally scalable via Docker replicas.
NFR-04	Security	No secrets in Docker images; model weights baked in at build time.
NFR-05	Maintainability	Typed Python 3.11 codebase, `flake8` clean, full docstrings.
NFR-06	Observability	All API calls and inference latencies tracked in Prometheus.

1.3 Success Metrics

Table 3: Success Metrics at Business, Model, and System Levels

Level	Metric	Target	Tool
Business	Prediction Throughput	> 100 req/s	Locust / Grafana
Business	ABSA Coverage	Aspects on $> 80\%$ of inputs	Prometheus
Model	Accuracy (test set)	> 0.85	`src.model.evaluate`
Model	F1 Macro (test set)	> 0.80	MLflow
System	P95 Latency	< 200 ms	Prometheus histogram
System	5xx Error Rate	$< 1\%$	Prometheus counter
System	Avg Inference Latency	< 500 ms	Prometheus histogram

1.4 Scope & Constraints

- **In Scope:** Real-time sentiment + ABSA inference, SHAP explainability, batch CSV processing, full observability stack, data quality validation, MLflow experiment tracking, PEFT/LoRA fine-tuning pipeline for Sentiment and Sarcasm tasks, multilingual support (English & Vietnamese).
- **Out of Scope:** User authentication / rate limiting, online learning / continuous retraining.
- **Constraints:** CPU-only Docker inference image; offline model serving (ONNX weights cached at build time); Python 3.11; GPU (e.g., L4) required for fine-tuning.

2 System Design & Architecture (15%)

2.1 High-Level Architecture

The system consists of five containerised services orchestrated by Docker Compose on a shared bridge network (`sentiment-network`). Figure 1 shows the component diagram.

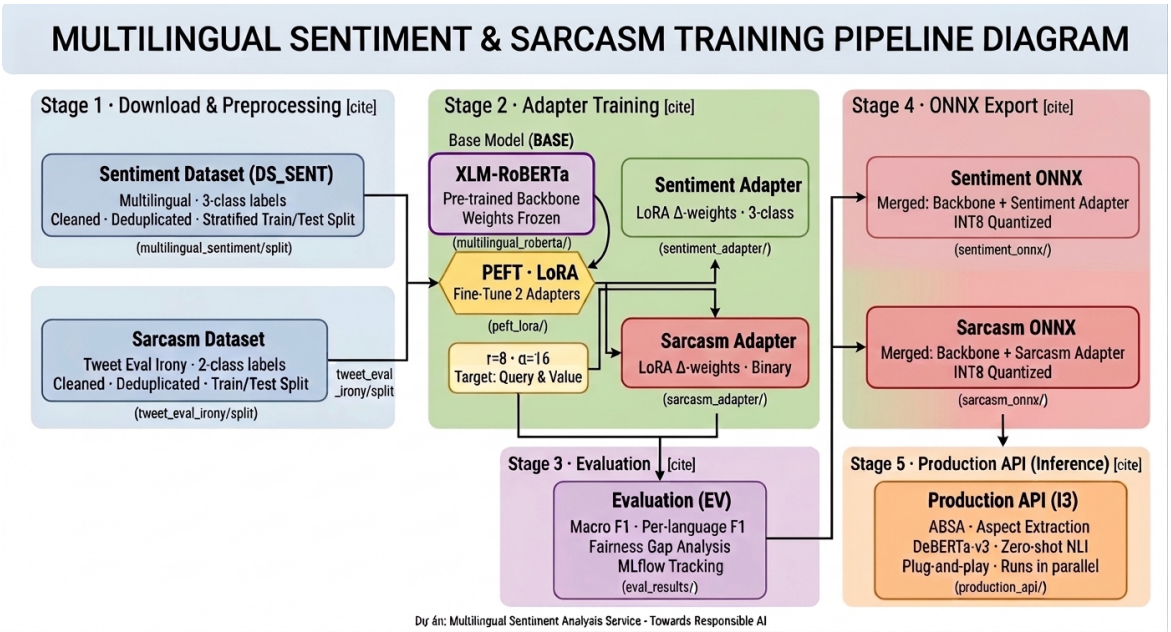

Figure 1: High-level component diagram – Sentiment Analysis Service.

2.2 Component Design & Responsibilities

Table 4: Service Responsibilities

Service	Image	Responsibility
`fastapi_app`	python:3.11-slim (custom)	Core ML inference, REST API, metrics exposure
`frontend`	Angular/Nginx	Chatbot-style web UI for sentiment queries
`prometheus`	prom/prometheus:v2.45.0	Time-series metrics scraping, alerting rules
`grafana`	grafana/grafana:10.0.0	Dashboard visualisation, alert notification
`mlflow`	ghcr.io/mlflow:v2.4.1	Experiment run tracking, artifacts storage

Table 5: Resource Limits per Service (Docker Compose)

Service	CPU Limit	Memory Limit
fastapi_app	1.50 vCPU	4 GB
frontend	0.25 vCPU	512 MB
prometheus	–	256 MB
grafana	–	512 MB
mlflow	0.50 vCPU	1 GB

2.3 Data Flow

1. User submits text to `POST /predict` (or `/explain`) via the Angular frontend or any HTTP client.
2. FastAPI validates the request body through Pydantic schemas (`PredictRequest`).
3. Validated text is forwarded to `BaselineModelInference.predict_single()`.
4. The XLM-RoBERTa tokenizer encodes the text; the ONNX-optimised model with merged LoRA weights returns a softmax output giving sentiment label and confidence.
5. The ABSA module extracts aspect-level sentiments (food, service, ambiance, price, location, general) above a 0.5 threshold.
6. For `/explain`, a SHAP `Explainer` returns per-token attribution scores.
7. The response is serialised by `PredictResponse` and returned to the client.
8. The monitoring middleware records request counts and latencies for Prometheus.

2.4 Technology Stack Justification

Table 6: Technology Decisions and Trade-off Analysis

Layer	Choice	Justification	Trade-off
API	FastAPI	Async-native, auto-generates OpenAPI/Swagger, Pydantic integration	Node.js higher throughput for pure I/O
ML Model	XLM-RoBERTa	Strong cross-lingual backbone; supports English and Vietnamese; backbone shared across all tasks	Larger than DistilBERT; mitigated by ONNX + INT8 quantization
Fine-tuning	PEFT / LoRA	Parameter-efficient; separate lightweight adapters for Sentiment and Sarcasm; multiple adapters stackable on a single backbone to minimise memory footprint	Requires adapter management logic; mitigated by DVC pipeline
Container	Docker multi-stage	Smaller image: builder installs deps, runtime copies site-packages only	Longer initial build time
Tracking	MLflow	Open-source, self-hosted, native DVC integration	W&B has a richer hosted UI
Observability	Prometheus + Grafana	Industry standard, battle-tested, no vendor lock-in	Higher operational complexity vs. SaaS
Data Versioning	DVC	Git-native, reproducible pipelines, remote storage	Steeper learning curve than plain scripts
CI/CD	GitHub Actions	Free for public repos, tight GitHub integration	Jenkins more flexible for complex pipelines

3 Implementation (40%)

3.1 ML Pipeline (15%)

3.1.1 Datasets & Machine Learning Models

The system integrates diverse data sources and advanced modelling techniques to support multiple text-classification tasks:

- **Datasets:**
  - **ABSA SemEval 2014 (Restaurants):** Provides sentence and aspect-level sentiment annotations. Used for baseline evaluation.
  - **Sarcasm Dataset (`tweet_eval_irony_v1`):** Identifies irony and sarcasm in social media contexts.
  - **Multilingual Sentiment (`multilingual_sentiment_v1`):** English and Vietnamese data for training multilingual capabilities.
- **Machine Learning Models:**
  - **Base Model (`xlm-roberta-base`):** Serves as a unified backbone for superior cross-lingual understanding.
  - **Training Strategy (PEFT/LoRA):** Parameter-Efficient Fine-Tuning via Low-Rank Adaptation (LoRA) is employed. Separate lightweight adapters are trained for Sentiment Analysis and Sarcasm Detection, allowing multiple adapters to be stacked efficiently during inference to minimise memory footprint.
  - **Class Imbalance Handling:** To address significant class imbalances (e.g., the minority "neutral" class in Vietnamese datasets), the system incorporates a `WeightedLossTrainer` that dynamically computes and applies class-weighted Cross-Entropy Loss, alongside optional Random Oversampling during dataset construction.
  - **Hardware-Aware Optimisation:** The training pipeline automatically detects available accelerators (CUDA, MPS, CPU). On supported GPUs (e.g., L4), it automatically enables Mixed Precision (`bf16`), fused optimisers (`adamw_torch_fused`), pinned memory, and tuned Data Loader workers to maximise training throughput.
  - **Inference Optimisation (ONNX & Quantization):** HuggingFace models are exported to ONNX format (both FP32 and INT8 quantization), significantly reducing inference latency and resource consumption in production.

3.1.2 Reproducible DVC Pipeline

The complete lifecycle of data and models is orchestrated via Data Version Control (DVC). The pipeline is defined as a Directed Acyclic Graph (DAG) in `dvc.yaml` to ensure strict reproducibility:

1. **Data Engineering:**
   - `download`: Automatically retrieves raw data from external sources.
   - `preprocess`: Executes a configurable transformation chain (label mapping, sentiment derivation, cleaning, deduplication, length filtering, and Train/Test splitting according to `params.yaml`). Includes class distribution analysis and oversampling strategies for minority classes.
   - `validate`: Enforces data quality checks (null ratios, minimum samples, label distributions) and automatically generates `quality_report.json` and `class_balance_report.json`.
2. **Modeling:**
   - `download_sarcasm` / `download_sentiment`: Prepares task-specific training datasets.
   - `finetune_sarcasm` / `finetune_sentiment`: Triggers LoRA adapter training.
   - `evaluate_baseline` / `evaluate_finetuned`: Evaluates models on test splits, generating detailed reports: `metrics_finetuned.json`, `per_language_f1.json`, and `fairness_report.json`.
3. **Deployment Preparation:**
   - `export_onnx_sentiment` / `export_onnx_sarcasm`: Merges LoRA weights into the base model and exports ONNX graphs.
   - `benchmark_onnx`: Measures real-world inference performance, yielding `onnx_benchmark.json`.

Figure 2 illustrates the DAG of the data pipeline generated by DVC.

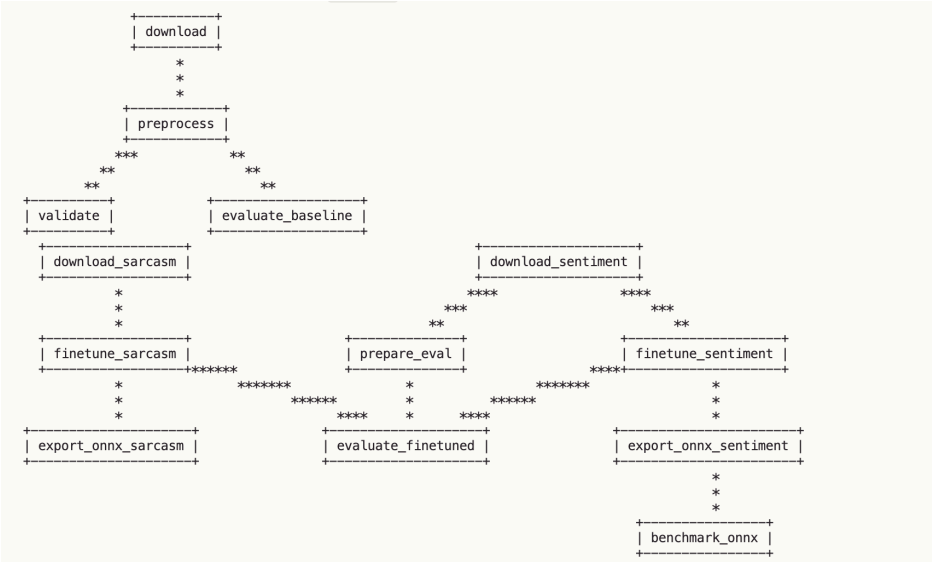

Figure 2: DVC pipeline DAG.

3.1.3 Experiment Tracking with MLflow

MLflow is deeply integrated into the source code to track the entire process from data validation to model training:

- **Data Quality Tracking:** The Data Validation process automatically logs metrics directly to the `data_preprocessing` experiment. Metrics include `total_samples`, split-specific null ratios, and a `passed_quality_checks` flag. The `quality_report.json` is attached as an artifact.
- **Model Training Tracking:**
  - Integration with HuggingFace `Trainer` (`report_to=["mlflow"]`) automatically logs loss, learning rate, and metrics per epoch.
  - **Organisational Structure:** Runs are organised into task-specific experiments (e.g., `phase2_finetuning_sentiment`, `phase2_finetuning_sarcasm`).
  - **Automatic Tagging:** Each run is explicitly tagged with `task`, `git_sha` (linking model version to code), `device`, and `dataset_version`.
  - **Hyperparameter Tracking:** Parameters such as `learning_rate`, `batch_size`, and `epochs` are automatically ingested from the centralised `params.yaml` and logged for comparative analysis.

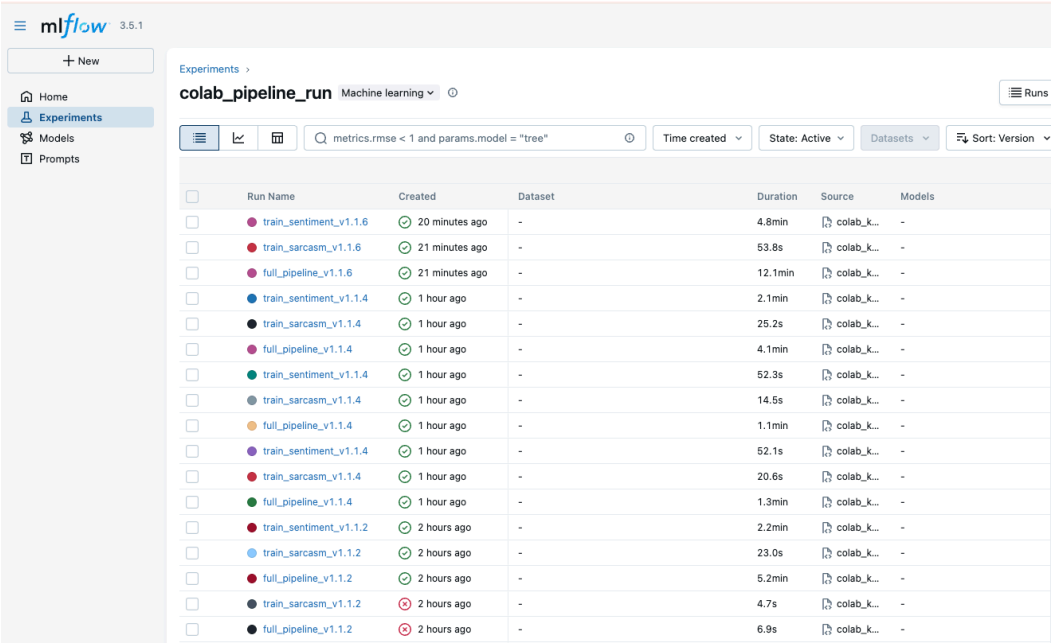

Figure 3: MLflow dashboard overview on DagsHub.

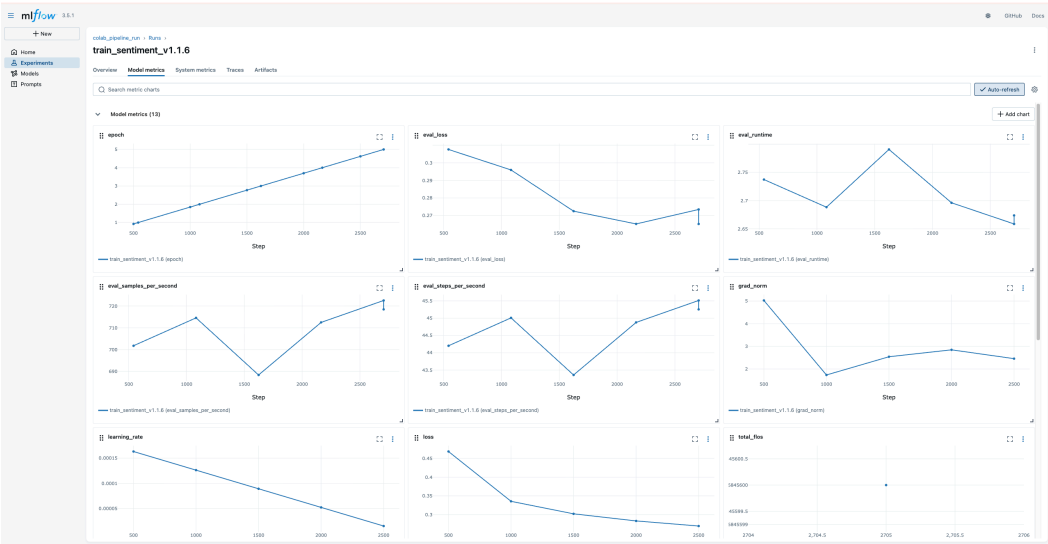

Figure 4: Detailed experiment run metrics tracked in MLflow.

3.1.4 Model Evaluation & Fine-tuning Results

The fine-tuning phase yields significant improvements, especially for minority classes, owing to the class imbalance handling strategies employed (e.g., WeightedLossTrainer).

Figure 5 presents the classification report detailing precision, recall, and F1-score across all sentiment classes. Figure 6 illustrates the overall confusion matrix, demonstrating the model's robust predictive performance and its ability to correctly identify the challenging "neutral" class.

Classification Report (Finetuned) — Sentiment

	precision	recall	f1-score	support
positive	0.9646	0.8846	0.9229	1880
negative	0.9369	0.9005	0.9184	1699
neutral	0.5434	0.8074	0.6496	457
macro avg	0.8150	0.8642	0.8303	4036
weighted avg	0.9053	0.8826	0.8900	4036

Figure 5: Classification report evaluating the fine-tuned model across all sentiment classes.

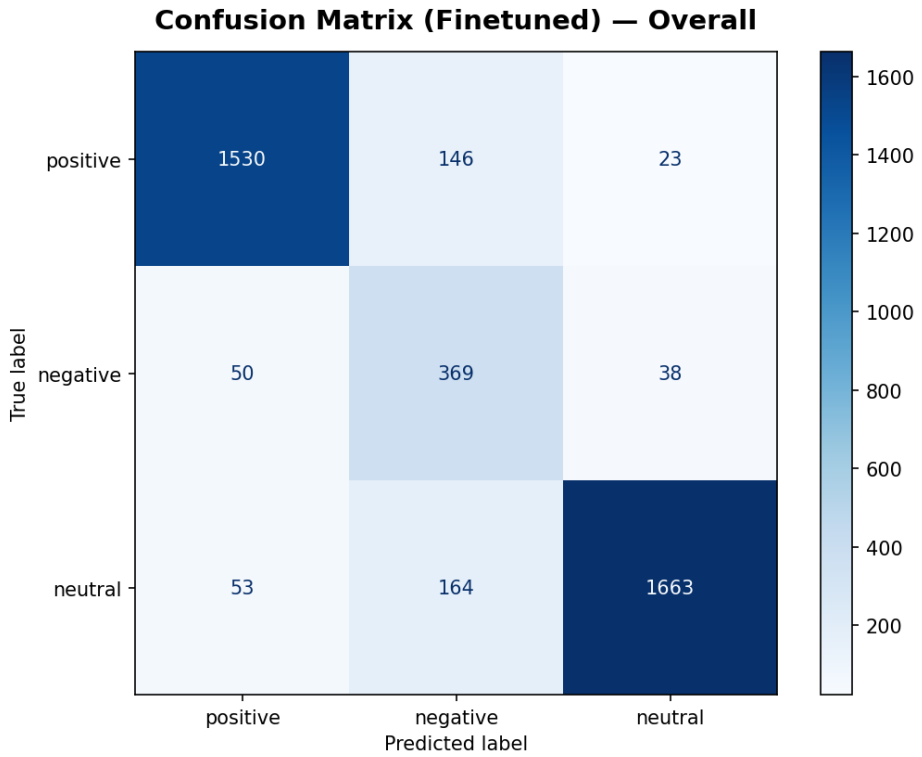

Figure 6: Overall confusion matrix of the fine-tuned sentiment analysis model.

3.2 Deployment (15%)

3.2.1 REST API Design

The FastAPI application (`src/main.py`) provides a self-documenting REST interface:

Table 7: API Endpoint Reference

Method	Endpoint	Description
GET	`/health`	Liveness probe; returns model load status and supported languages
POST	`/predict`	Single-text sentiment + ABSA inference
POST	`/explain`	SHAP token-level attributions for a single text
POST	`/batch_predict`	CSV upload (`multipart/form-data`); returns job ID
GET	`/batch_status/{job_id}`	Polls processing status of a batch job
GET	`/metrics`	Prometheus-compatible plain-text metric exposition

Error Handling: Custom exception handlers return appropriate HTTP status codes:

- `UnsupportedLanguageError` → HTTP 400 plain-text message.
- `ModelError` → HTTP 500 plain-text message.
- Model not yet loaded → HTTP 503 via the `get_model` dependency.

Request and response schemas are enforced via Pydantic v2 with field-level validation:

```python
class PredictRequest(BaseModel):
    text: str = Field(min_length=1, max_length=2000)
    lang: str = "en"

class PredictResponse(BaseModel):
    text: str
    sentiment: Literal["positive", "negative", "neutral"]
    confidence: float = Field(ge=0.0, le=1.0)
    aspects: list[AspectSentimentOut] = Field(default_factory=list)
    sarcasm_flag: bool
    latency_ms: float

class ExplainResponse(BaseModel):
    tokens: list[str]
    shap_values: list[float]
    base_value: float
    latency_ms: float

    @model_validator(mode="after")
    def validate_lengths(self):
        if len(self.tokens) != len(self.shap_values):
            raise ValueError("tokens and shap_values must match in length")
        return self
```

Listing 1: Core Pydantic schemas (contracts/schemas.py)

Figure 7 shows the chatbot-style web interface in action, demonstrating both the prediction and SHAP explanation capabilities via the REST API.

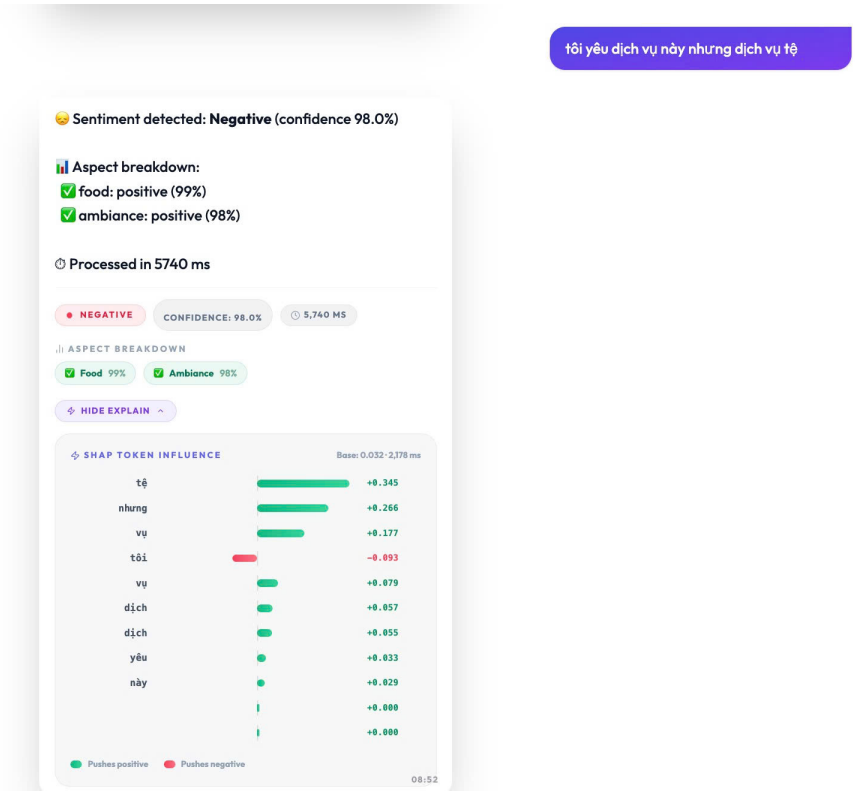

Figure 7: Web UI demonstrating sentiment prediction and SHAP explanation.

3.2.2 Docker Containerisation

The Dockerfile employs a **multi-stage build** to minimise the final image size:

- **Stage 1 (builder):** `python:3.11-slim` with `build-essential`; installs PyTorch (CPU wheel) and all `requirements.txt` packages.
- **Stage 2 (runtime):** copies only `site-packages` and `bin` from the builder, plus application source.
- **Model caching:** `RUN python src/model/download_models.py` bakes the HuggingFace weights into the image at build time.
- **Offline enforcement:** `TRANSFORMERS_OFFLINE=1` and `HF_DATASETS_OFFLINE=1` eliminate network calls at runtime.

```
# Stage 1: build
FROM python:3.11-slim AS builder
RUN pip install torch --index-url https://download.pytorch.org/whl/cpu \
```

```
 && pip install -r requirements.txt

# Stage 2: lean runtime
FROM python:3.11-slim
COPY --from=builder /usr/local/lib/python3.11/site-packages ...
COPY src/ contracts/
RUN python src/model/download_models.py
ENV TRANSFORMERS_OFFLINE=1
CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Listing 2: Dockerfile multi-stage build summary

3.2.3 Docker Compose Orchestration

Five services are co-orchestrated in `docker-compose.yml` on the `sentiment-network` bridge:

```
services:
  fastapi_app:   # depends_on: mlflow
  frontend:      # depends_on: fastapi_app
  prometheus:    # mounts alert_rules.yml + prometheus.yml
  grafana:       # depends_on: prometheus; mounts provisioning/
  mlflow:        # standalone; experiment tracking UI
```

Listing 3: Service graph in docker-compose.yml

All services share `sentiment-network`; inter-service DNS allows e.g. `http://mlflow:5000`.

3.3 Monitoring (10%)

3.3.1 Prometheus Metrics

Three Prometheus metric families are defined in `src/monitoring/metrics.py`:

Table 8: Prometheus Metric Definitions

Metric Name	Type	Description	Labels
`api_requests_total`	Counter	Total number of API requests	method, endpoint, http_status
`api_request_latency_seconds`	Histogram	End-to-end request latency	method, endpoint
`model_inference_latency_seconds`	Histogram	Model forward-pass duration	–

The `monitor_middleware` ASGI middleware wraps every request, computing wall-clock duration and incrementing both counters before returning the response.

3.3.2 Grafana Dashboards

Grafana is provisioned automatically via the `infra/grafana/provisioning/` volume mount, with:

- Datasource provisioning pointing to the internal Prometheus service.
- Pre-built dashboard JSON files visualising request rate, latency percentiles, and error-rate time series.

3.3.3 Alerting Rules

Two alert rules are defined in `infra/prometheus/alert_rules.yml`:

```yaml
- alert: HighErrorRate
  expr: rate(api_requests_total{http_status=~"5.."}[5m])
      / rate(api_requests_total[5m]) > 0.1
  for: 1m
  labels: { severity: critical }
  annotations:
    summary: "High HTTP 5xx error rate on {{ $labels.endpoint }}"
    description: "Error rate above 10% for 1 minute."

- alert: HighInferenceLatency
  expr: model_inference_latency_seconds_sum
      / model_inference_latency_seconds_count > 0.5
  for: 5m
  labels: { severity: warning }
  annotations:
    summary: "High model inference latency"
    description: "Average inference latency above 500ms for 5 minutes."
```

Listing 4: Prometheus alerting rules

4 Testing & CI/CD (15%)

4.1 Test Strategy

The test suite in `tests/` is structured around three tiers:

Table 9: Test Coverage Summary

Test Type	Location	Coverage
Unit – Data Transforms	`tests/data/`	Each transform (TextCleaner, LengthFilter, DuplicateRemover, Splitter, SentimentDeriver) tested in isolation with mock DataFrames
Unit – Model / Contracts	`tests/model/`, `tests/contracts/`	ModelConfig defaults, schema field constraints, model_validator logic
Integration – API	`tests/test_api.py`	All five endpoints tested via FastAPI TestClient

4.1.1 Integration Test Examples

```python
def test_predict(client):
    response = client.post(
        "/predict",
        json={"text": "I love this product!", "lang": "en"}
    )
    assert response.status_code == 200
    data = response.json()
    assert "sentiment" in data   # overall sentiment label
    assert "confidence" in data  # float in [0, 1]
    assert "aspects" in data     # ABSA aspect list

def test_explain(client):
    response = client.post(
        "/explain",
        json={"text": "I love this product!", "lang": "en"}
    )
    assert response.status_code == 200
    data = response.json()
    assert "tokens" in data
    assert "shap_values" in data
    # Schema invariant enforced by Pydantic model_validator
    assert len(data["tokens"]) == len(data["shap_values"])
```

Listing 5: Integration tests (tests/test_api.py)

4.2 CI/CD Pipeline

The GitHub Actions workflow (`.github/workflows/ci.yml`) triggers on every push to `main`/`develop` and on pull-request to `main`:

```yaml
on:
```

```yaml
  push:         { branches: [main, develop] }
  pull_request: { branches: [main] }

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v4
        with: { python-version: '3.11' }
      - run: pip install -r requirements.txt
      - run: pytest --cov=src tests/
      - run: pip install flake8
      - run: flake8 . --select=E9,F63,F7,F82
      - run: flake8 . --exit-zero --max-complexity=10
```

Listing 6: GitHub Actions CI/CD configuration

Pipeline stages:

1. **Checkout** – fetches source code on `ubuntu-latest`.
2. **Dependency Install** – installs all packages including `pytest`, `pytest-cov`, `httpx`.
3. **Test & Coverage** – runs full suite; `pytest-cov` measures line coverage across `src/`.
4. **Lint (hard)** – `flake8` fails the build on syntax errors and undefined names.
5. **Lint (soft)** – `flake8` in warning mode for complexity and line-length metrics.

5 Responsible AI (10%)

5.1 Model Explainability with SHAP

The `/explain` endpoint exposes token-level feature attributions using the **SHAP** library (`shap` $\geq$ `0.42.0`).

5.1.1 Technical Implementation

```python
def get_shap_explanation(self, text: str, lang: str = "en") -> SHAPResult:
    import shap
    self._check_language(lang)
    # Identify the predicted class index
    predicted_probs     = self._predict_probabilities(text)[0]
    predicted_class_idx = int(predicted_probs.argmax().item())
    # Wrap HuggingFace pipeline in SHAP Explainer
    pipe        = self._get_classification_pipeline()
```

```
    explainer   = shap.Explainer(pipe)
    shap_values = explainer([text])
    # Extract token strings and SHAP scores for the predicted class
    raw_tokens = shap_values.data[0]
    tokens = raw_tokens.tolist() if hasattr(raw_tokens, "tolist") else list(raw_tokens)
    values = shap_values.values[0][:, predicted_class_idx].tolist()
    base   = float(shap_values.base_values[0][predicted_class_idx])
    return SHAPResult(tokens=tokens, shap_values=values, base_value=base)
```

Listing 7: SHAP explanation in BaselineModelInference

The explanation workflow:

1. The text is classified to determine the predicted class index.
2. `shap.Explainer` computes Shapley values by marginalising over permutations of input tokens.
3. Per-token SHAP scores for the predicted class are extracted alongside the base value.
4. The `ExplainResponse` schema enforces `len(tokens) == len(shap_values)` via `model_validator`.

Figure 8 and Figure 9 demonstrate SHAP waterfall plots for specific test samples, highlighting the token-level sentiment attributions.

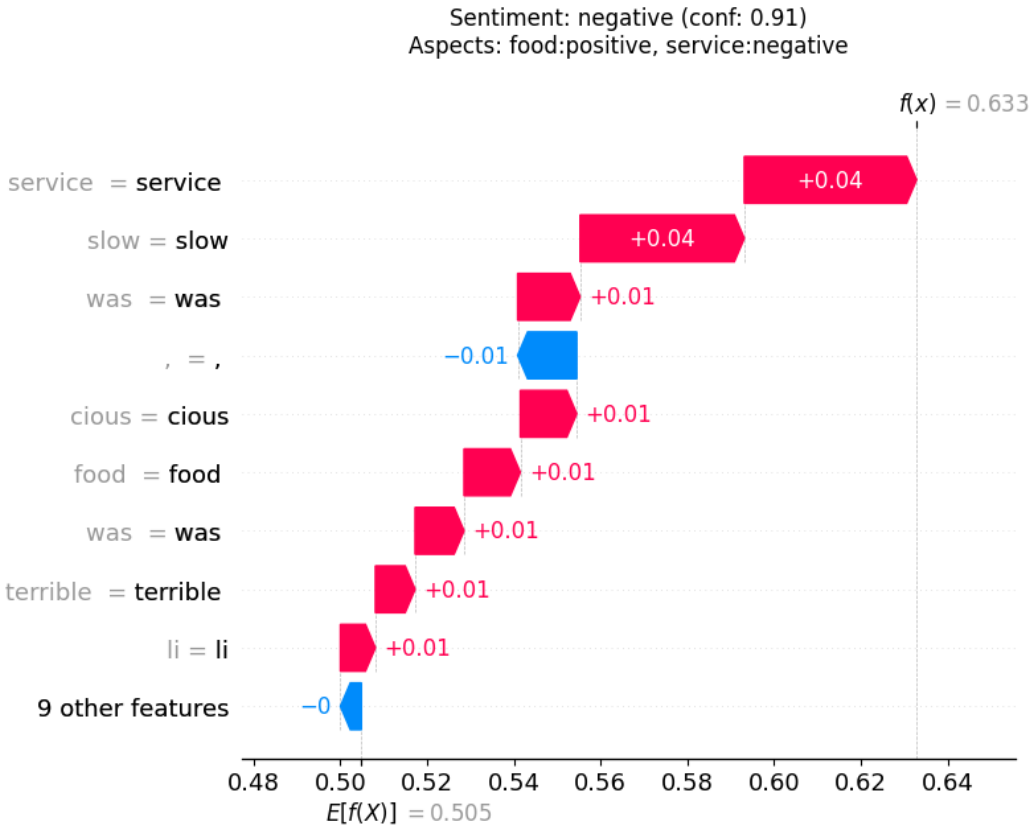

Figure 8: SHAP explanation highlighting negative aspects regarding food and service.

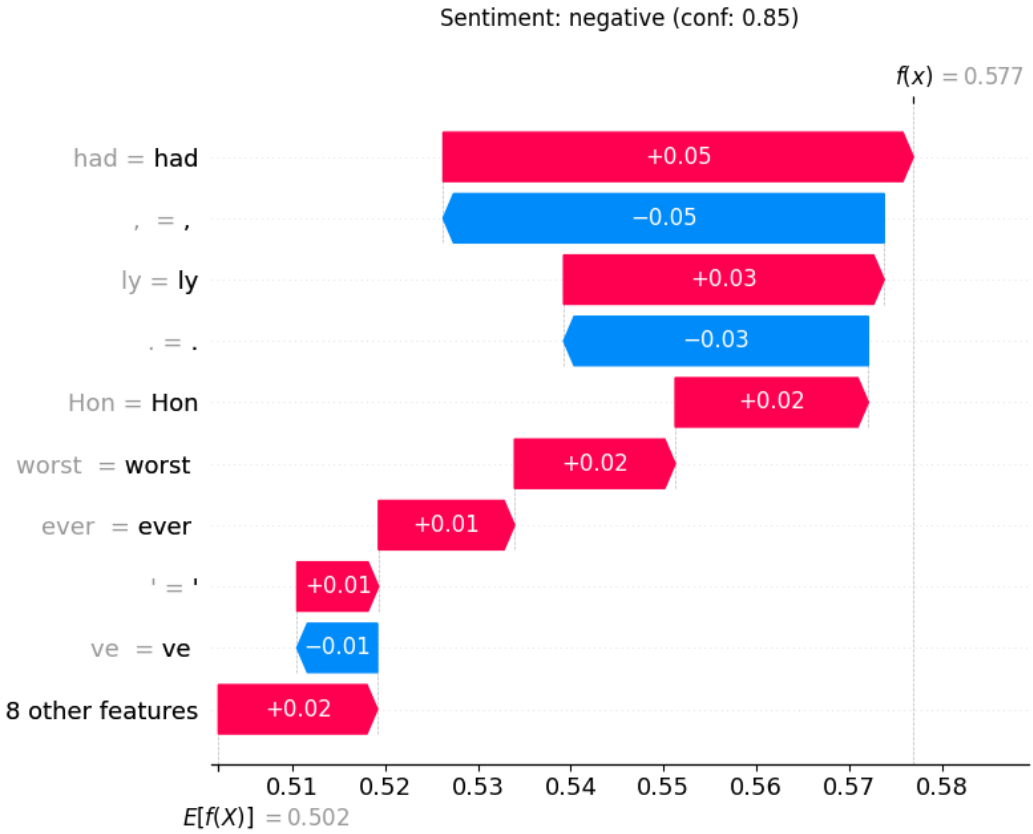

Figure 9: SHAP explanation indicating a negative sentiment prediction.

5.2 Fairness Analysis

Known Biases. The `xlm-roberta-base` model, fine-tuned via LoRA on the `multilingual_sentiment_v1` dataset, may exhibit the following biases:

- **Domain bias:** Training data skews toward social-media and review text; performance may degrade on highly formal or technical language.
- **Language imbalance:** English training samples may outnumber Vietnamese samples, potentially reducing fairness across languages. Per-language F1 scores are tracked in `per_language_f1.json` and logged to MLflow.
- **Sarcasm:** The dedicated LoRA sarcasm adapter (trained on `tweet_eval_irony_v1`) addresses this limitation; the `sarcasm_flag` field in the API response is populated accordingly.

Dataset Bias Mitigation. The `DataQualityValidator` emits distribution warnings when any class falls below 5% in a split, enabling early detection of imbalance before evaluation.

5.3 Data Privacy Considerations

- No user input data is persisted; the service is **stateless** – all processing is in-memory per request.
- No PII is logged by the monitoring middleware; only HTTP method, path, status code, and timing are recorded.
- Model weights are stored inside the Docker image; no external registry calls occur at runtime.

5.4 Ethical Implications

Table 10: Ethical Risk Register

Risk	Implication & Mitigation	Status
Biased training data	XLM-RoBERTa fine-tuned on multilingual sentiment and irony data; may disadvantage highly formal text or underrepresented language variants. Per-language fairness tracked via `fairness_report.json`.	Acknowledged
Misclassification harm	Incorrect label could negatively influence business decisions. SHAP explanations and human-in-the-loop review are recommended.	SHAP done
Sarcasm detection	Missed sarcasm leads to reversed polarity. Dedicated LoRA sarcasm adapter trained on `tweet_eval_irony_v1`; `sarcasm_flag` populated in API response.	Implemented
Job displacement	May reduce manual review roles. System is positioned as decision-support, not a replacement.	Discussed

6 Documentation (10%)

6.1 Repository Structure

```
Sentiment-Analysis-Service/
  .github/workflows/ci.yml     # GitHub Actions CI pipeline
  app/                         # Angular frontend + Dockerfile
  contracts/                   # Shared interfaces, schemas, errors
    model_interface.py         # Abstract ModelInference base class
    schemas.py                 # Pydantic API schemas
    errors.py                  # Custom exceptions
  data/                        # DVC-managed data artefacts
```

```
    external/, raw/, processed/, reports/
  infra/
    prometheus/               # prometheus.yml, alert_rules.yml
    grafana/provisioning/     # Datasource + dashboard JSON
  src/
    main.py                   # FastAPI app entrypoint
    api/, core/, services/    # Routers, logic, helpers
    data/                     # Data pipeline & transforms
    model/                    # Baseline model, config, evaluate
    monitoring/metrics.py     # Prometheus metrics + middleware
  tests/                      # Pytest test suite
  Dockerfile                  # Multi-stage Docker build
  docker-compose.yml          # Five-service orchestration
  dvc.yaml                    # Reproducible ML pipeline stages
  params.yaml                 # Experiment & pipeline parameters
  requirements.txt            # Python dependencies (pinned ranges)
  README.md                   # Setup & usage guide
  ARCHITECTURE.md             # System design documentation
```

Listing 8: Top-level repository layout

6.2 API Documentation

FastAPI auto-generates a fully interactive **Swagger UI** at `http://localhost:8000/docs` and a **ReDoc** interface at `http://localhost:8000/redoc` from the Pydantic schemas and endpoint decorators, satisfying the OpenAPI/Swagger documentation requirement without additional tooling.

6.3 Setup & Deployment Guide

```
# Clone the repository
git clone <repo-url>
cd Sentiment-Analysis-Service

# Launch full stack
docker-compose up --build

# Services:
#   API         http://localhost:8000/docs
#   Grafana     http://localhost:3000    (admin / admin)
#   MLflow      http://localhost:5005
#   Prometheus  http://localhost:9091

# Run tests locally (Python 3.11 environment required)
pip install -r requirements.txt
pytest --cov=src tests/
```

```

# Re-run DVC pipeline (data + preprocess + validate + evaluate)
dvc repro
```

Listing 9: Quick-start commands

6.4 Code Quality

- All public functions and classes carry **docstrings**.
- **Type hints** throughout (`from __future__ import annotations`).
- Immutable configuration via `@dataclass(frozen=True)` in `ModelConfig`.
- Separation of concerns: `contracts/` holds shared interfaces; business logic in `src/`; infrastructure in `infra/`.
- All modules are **flake8-clean** (E9/F-series hard errors pass in CI).

7 Conclusion

The Sentiment Analysis Service successfully demonstrates a complete, production-grade ML system lifecycle:

1. **Problem Definition:** Clearly motivated by business need; success metrics defined at business, model, and system levels.

2. **Architecture:** Microservices design with full observability and experiment tracking, all resource-limited via Docker Compose.

3. **ML Pipeline:** Reproducible data pipeline (DVC) encompassing three datasets (SemEval-2014, `tweet_eval_irony_v1`, `multilingual_sentiment_v1`), data-quality validation, PEFT/LoRA fine-tuning of `xlm-roberta-base` for Sentiment and Sarcasm tasks, ONNX export with INT8 quantization, and MLflow-tracked evaluations including per-language fairness reporting.

4. **Deployment:** Multi-stage Docker image with offline ONNX model serving; REST API with Pydantic schemas; health checks and CORS middleware.

5. **Monitoring:** Three Prometheus metrics via ASGI middleware; Grafana dashboards auto-provisioned; two alerting rules for error rate and inference latency.

6. **Testing & CI/CD:** Three test layers (unit, integration, data quality); GitHub Actions lints and tests on every commit.

7. **Responsible AI:** SHAP token attributions via `/explain`; per-language fairness analysis; privacy-preserving stateless design; ethical risk register.

A Full Dependency List

```
dvc>=3.0,<4.0
mlflow>=2.0,<3.0
pydantic>=2.0,<3.0
fastapi>=0.110.0
uvicorn>=0.27.0
prometheus-client>=0.20.0
python-multipart>=0.0.9
PyYAML>=6.0,<7.0
scikit-learn>=1.0,<2.0
pytest>=8.0
pytest-cov>=5.0
httpx>=0.27.0
pandas>=2.2.0
python-dotenv>=1.0.1
torch>=2.0.0
transformers>=4.30.0
shap>=0.42.0
matplotlib>=3.7.0
```

Listing 10: requirements.txt

B Individual Contribution

Table 11: Team Member Responsibilities

Member	**Primary Contributions**
Dương Hồng Quân	Monitoring stack (Prometheus + Grafana), alerting rules, data validation (`src/data/validators.py`), testing
Phạm Đức Long	FastAPI application, Pydantic schemas, Docker containerisation, CI/CD pipeline (GitHub Actions)
Đỗ Quốc Trung	ML pipeline design, model evaluation (`src/model/`), MLflow integration, DVC pipeline configuration